

Random Forests

Random Forests: Mathematical Foundation and Assumptions

Introduction

Random forests are an ensemble learning method that combines many decision trees to produce a single predictive model. They are used for both regression and classification tasks and are particularly effective when relationships in the data are nonlinear or involve complex interactions.

The key idea is to reduce variance by averaging many decorrelated models.

Decision Trees as Base Learners

A random forest is built from decision trees. Each tree partitions the feature space into regions and makes predictions based on those regions.

Regression Trees

For regression, a tree partitions the feature space into regions ($R_1, R_2, \dots, R_J$), and predicts:

$$\hat{f}(x) = \sum_{j=1}^J c_j \mathbf{1}(x \in R_j)$$

where (c_j) is typically the mean of the response values in region (R_j).

Splits are chosen to minimize **residual sum of squares (RSS)**:

$$\sum_{i: x_i \in R_1} (y_i - \bar{y}_{R_1})^2 + \sum_{i: x_i \in R_2} (y_i - \bar{y}_{R_2})^2$$

Classification Trees

For classification, trees predict class labels using majority vote within each region.

Splits are chosen to reduce impurity, commonly using:

- **Gini Index:**

$$G = \sum_{k=1}^K p_k (1 - p_k)$$

- **Entropy:**

$$H = - \sum_{k=1}^K p_k \log p_k$$

where (p_k) is the proportion of class (k) in a node.

Random Forest Algorithm

A random forest builds (B) trees using two sources of randomness:

1. Bootstrap Sampling

For each tree ($b = 1, \dots, B$), draw a bootstrap sample from the data.

2. Random Feature Selection

At each split, select a random subset of (m) features (with ($m < p$)) and choose the best split only among those features.

Prediction

Regression

$$\hat{f}(x) = \frac{1}{B} \sum_{b=1}^B T_b(x)$$

Classification

$$\hat{C}(x) = \text{mode}\{T_1(x), T_2(x), \dots, T_B(x)\}$$

Why Random Forests Work (Bias-Variance Tradeoff)

Let each tree have variance (σ^2) and pairwise correlation (ρ). Then the variance of the ensemble is approximately:

$$\text{Var}(\hat{f}) \approx \rho \sigma^2 + \frac{1-\rho}{B} \sigma^2$$

Key insight: - Increasing (B) reduces variance - Reducing correlation (ρ) (via feature randomness) is crucial

Assumptions

Random forests make fewer assumptions than parametric models, but still rely on:

1. IID Observations

Training data are assumed to be independent and identically distributed.

2. Stationarity of Distribution

Training and test data come from the same underlying distribution.

3. Sufficient Signal in Features

Predictors must contain information relevant to the response.

4. No Strong Extrapolation Requirement

Random forests do not extrapolate well beyond observed data.

Out-of-Bag (OOB) Error Estimation

Each tree is trained on a bootstrap sample, leaving about ($1/3$) of observations unused.

For each observation, predictions from trees where it was not included are averaged:

$$\hat{f}_{\text{OOB}}(x_i) = \frac{1}{|\mathcal{B}_i|} \sum_{b \notin \mathcal{B}_i} T_b(x_i)$$

This provides an unbiased estimate of generalization error.

Shortcomings

1. Lack of Interpretability

- Difficult to interpret compared to linear regression or a single tree
- Feature importance measures can be misleading

2. Computational Cost

- Training many trees can be slow and memory-intensive

3. Bias in High-Dimensional Sparse Settings

- Can struggle when the number of irrelevant features is large

4. Poor Extrapolation

- Predictions are averages of observed outcomes → cannot extrapolate beyond training range

5. Correlated Features

- Highly correlated predictors can reduce effectiveness of feature importance measures

Summary

Random forests improve upon decision trees by: - Reducing variance through averaging - Reducing correlation via random feature selection

They are a flexible, powerful tool that performs well in practice with minimal tuning, making them a strong choice for both regression and classification tasks (such as with the `iris` dataset in R).